

```

import { useState, useEffect, useCallback } from "react";

// — Constants —————
const MAIN_INTERVALS      = [0, 1, 3, 6, 13, 27];
const LIGHT_INTERVALS     = [60, 120, 180, 365];
const MAIN_STAGE_LABELS   = ["J0", "J1", "J3", "J6", "J13", "J27"];
const LIGHT_STAGE_LABELS  = ["J60", "J120", "J180", "J365"];
const MAX_SLOTS = 6;
const MAX_NEW    = 2;
const STORAGE_KEY = "medical-srs-v3";

// — Helpers —————
function dateKey(date) { return date.toISOString().slice(0, 10); }
function addDays(date, n) { const d = new Date(date); d.setDate(d.getDate() + n); }
function dayOfWeek(date) { return (date.getDay() + 6) % 7; }

const MONTHS   = ["January", "February", "March", "April", "May", "June", "July", "August", "September", "October", "November", "December"];
const WEEKDAYS = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"];

// — Scheduling —————
function findDayForPair(schedule, fromDate) {
  let d = new Date(fromDate);
  for (let i = 0; i < 400; i++) {
    const key = dateKey(d);
    const count = (schedule[key]?.main || []).length;
    if (count <= MAX_SLOTS - 2) return new Date(d);
    d = addDays(d, 1);
  }
  return null;
}

function schedulePair(schedule, id1, id2, startDate) {
  // Deep copy only what we need
  const s = {};
  for (const [k, v] of Object.entries(schedule)) {
    s[k] = { main: [...(v.main || [])], light: [...(v.light || [])], checked: { } };
  }

  // Main track – respects slot cap
  for (let si = 0; si < MAIN_INTERVALS.length; si++) {
    const target = addDays(startDate, MAIN_INTERVALS[si]);
    const actual = findDayForPair(s, target);
    if (!actual) continue;
    const key = dateKey(actual);
  }
}

```

```

    if (!s[key]) s[key] = { main: [], light: [], checked: {} };
    s[key].main.push({ lessonId: id1, stage: si });
    s[key].main.push({ lessonId: id2, stage: si });
  }

  // Light track – no cap, land exactly on target day
  for (let li = 0; li < LIGHT_INTERVALS.length; li++) {
    const target = addDays(startDate, LIGHT_INTERVALS[li]);
    const key = dateKey(target);
    if (!s[key]) s[key] = { main: [], light: [], checked: {} };
    s[key].light.push({ lessonId: id1, lightStage: li });
    s[key].light.push({ lessonId: id2, lightStage: li });
  }

  return s;
}

// — Colours —————
const mainColor = (stage) => [
  { bg: "#e8f5e9", text: "#2e7d32" },
  { bg: "#e3f2fd", text: "#1565c0" },
  { bg: "#fff8e1", text: "#f57f17" },
  { bg: "#f3e5f5", text: "#6a1b9a" },
  { bg: "#fce4ec", text: "#880e4f" },
  { bg: "#e0f2f1", text: "#00695c" },
  [stage] || { bg: "#eee", text: "#555" };
];

const lightColor = { bg: "#f9f5ff", text: "#7c3aed", border: "#ddd6fe" };

// — App —————
export default function App() {
  const today = new Date(); today.setHours(0,0,0,0);

  const [viewDate, setViewDate] = useState(new Date(today.getFullYear(), to
  const [schedule, setSchedule] = useState({});
  const [nextLesson, setNextLesson] = useState(1);
  const [selected, setSelected] = useState(null);
  const [loaded, setLoaded] = useState(false);
  const [toast, setToast] = useState(null);

  // — Persistence —————
  useEffect(() => {
    try {
      const raw = localStorage.getItem(STORAGE_KEY);
      if (raw) { const d = JSON.parse(raw); setSchedule(d.schedule || {}); setNex
    } catch (e) {}
    setLoaded(true);
  });

```

```
}, []);
```

```
useEffect(() => {  
  if (!loaded) return;  
  try { localStorage.setItem(STORAGE_KEY, JSON.stringify({ schedule, nextLesson  
}, [schedule, nextLesson, loaded]));
```

```
const showToast = (msg, err = false) => {  
  setToast({ msg, err });  
  setTimeout(() => setToast(null), 3000);  
};
```

```
// — Add lessons —————
```

```
const addLessons = useCallback((dateStr) => {  
  const date      = new Date(dateStr); date.setHours(0,0,0,0);  
  const day       = schedule[dateStr] || { main:[], light:[], checked:{} };  
  const newOnDay  = day.main.filter(e => e.stage === 0).length;  
  const total     = day.main.length;  
  const tKey      = dateKey(addDays(date, 1));  
  const tCount    = (schedule[tKey]?.main || []).length;  
  
  if (newOnDay >= MAX_NEW)      { showToast("Already 2 new lessons today.", t  
  if (total > MAX_SLOTS - 2)    { showToast("Day is full (6 slots).", true);  
  if (tCount > MAX_SLOTS - 2)  { showToast("Tomorrow too full for J1 rep.",  
  
  const id1 = nextLesson, id2 = nextLesson + 1;  
  setSchedule(schedulePair(schedule, id1, id2, date));  
  setNextLesson(nextLesson + 2);  
  showToast(`Lessons ${id1} & ${id2} added — all reps scheduled!`);  
}, [schedule, nextLesson]);
```

```
// — Undo —————
```

```
const undoLast = useCallback(() => {  
  if (nextLesson <= 1) return;  
  const id1 = nextLesson - 2, id2 = nextLesson - 1;  
  const ns = {};  
  for (const [k, v] of Object.entries(schedule)) {  
    const main  = (v.main || []).filter(e => e.lessonId !== id1 && e.lessonId  
    const light = (v.light || []).filter(e => e.lessonId !== id1 && e.lessonId  
    if (main.length || light.length) ns[k] = { main, light, checked: v.checked  
  }  
  setSchedule(ns);  
  setNextLesson(id1);  
  showToast(`Lessons ${id1} & ${id2} removed.`);  
}, [schedule, nextLesson]);
```

```
// — Toggle light rep checked —————
```

```

const toggleCheck = useCallback((dateStr, lessonId, lightStage) => {
  const checkKey = `${lessonId}-${lightStage}`;
  setSchedule(prev => {
    const day = prev[dateStr] || { main:[], light:[], checked:{} };
    const checked = { ...day.checked };
    checked[checkKey] = !checked[checkKey];
    return { ...prev, [dateStr]: { ...day, checked } };
  });
}, []);

// — Calendar —————
const firstDay = new Date(viewDate.getFullYear(), viewDate.getMonth(), 1);
const totalDays = new Date(viewDate.getFullYear(), viewDate.getMonth()+1, 0).ge
const startPad = dayOfWeek(firstDay);
const todayKey = dateKey(today);

const selDay      = selected ? (schedule[selected] || { main:[], light:[], check
const selMain     = selDay?.main || [];
const selLight    = selDay?.light || [];
const selChecked  = selDay?.checked || {};
const selNew      = selMain.filter(e => e.stage === 0);
const selReps     = selMain.filter(e => e.stage > 0);

const selCanAdd = selected && (() => {
  const day      = schedule[selected] || { main:[], light:[], checked:{} };
  const newOnDay = day.main.filter(e => e.stage === 0).length;
  const total    = day.main.length;
  const tKey     = dateKey(addDays(new Date(selected), 1));
  const tCount   = (schedule[tKey]?.main || []).length;
  return newOnDay < MAX_NEW && total <= MAX_SLOTS - 2 && tCount <= MAX_SLOTS -
  })();

const totalLessons = nextLesson - 1;

// Light rep completion stats
const totalLight = Object.values(schedule).reduce((a, d) => a + (d.light?.len
const doneLight = Object.values(schedule).reduce((a, d) => {
  const checked = d.checked || {};
  return a + (d.light || []).filter(e => checked[`${e.lessonId}-${e.lightStage}
}, 0);

if (!loaded) return (
  <div style={{ display:"flex", alignItems:"center", justifyContent:"center", h
});

return (
  <div style={{ fontFamily:"Georgia,serif", background:"#f5f0ea", minHeight:"10

```

```

    { /* Toast */
    {toast && (
      <div style={{ position:"fixed", top:16, left:"50%", transform:"translateX
        background: toast.err ? "#c62828" : "#2e7d32", color:"#fff",
        padding:"10px 22px", borderRadius:8, fontSize:14, zIndex:999,
        boxShadow:"0 4px 16px rgba(0,0,0,0.2)" }}>
        {toast.msg}
      </div>
    )}

    { /* Header */
    <div style={{ background:"#1a1208", color:"#f5f0ea", padding:"16px 28px", d
      <div>
        <div style={{ fontSize:10, letterSpacing:4, color:"#8c7f6e", marginBott
        <div style={{ fontSize:18 }}>Spaced Repetition Calendar</div>
        <div style={{ fontSize:11, color:"#8c7f6e", marginTop:2 }}>
          J0·J1·J3·J6·J13·J27 &nbsp;<span style={{ color:"#7c3aed" }}>+ J60·J12
        </div>
      </div>
      <div style={{ display:"flex", gap:10, flexWrap:"wrap" }}>
        {[
          { label:"Lessons", val: totalLessons, color:"#ef5
          { label:"Remaining", val: Math.max(0,250-totalLessons), color:"#ffa
          { label:"Light done", val: `${doneLight}/${totalLight}`, color:"#7c3
        ].map(({ label, val, color }) => (
          <div key={label} style={{ textAlign:"center", background:"rgba(255,25
            <div style={{ fontSize:20, fontWeight:"bold", color }}>{val}</div>
            <div style={{ fontSize:10, color:"#8c7f6e", letterSpacing:1 }}>{lab
          </div>
        )))
      </div>
    </div>

    { /* Progress bar */
    <div style={{ height:5, background:"#e0d9cf" }}>
      <div style={{ height:"100%", background:"linear-gradient(90deg,#c62828,#f
    </div>

    { /* Body */
    <div style={{ display:"flex", flex:1 }}>

      { /* — Calendar — */
      <div style={{ flex:1, padding:"20px 24px", overflowY:"auto", minWidth:0 }

        { /* Month nav */
        <div style={{ display:"flex", alignItems:"center", justifyContent:"spac

```

```

    <button onClick={() => setViewDate(new Date(viewDate.getFullYear(), v
    <div style={{ fontSize:18, color:"#1a1208" }}>{MONTHS[viewDate.getMon
    <button onClick={() => setViewDate(new Date(viewDate.getFullYear(), v
  </div>

  {/* Weekdays */}
  <div style={{ display:"grid", gridTemplateColumns:"repeat(7,1fr)", gap:
    {WEEKDAYS.map(d => <div key={d} style={{ textAlign:"center", fontSize
  </div>

  {/* Days */}
  <div style={{ display:"grid", gridTemplateColumns:"repeat(7,1fr)", gap:
    {Array.from({ length: startPad }).map((_,i) => <div key={`p${i}`} />)
    {Array.from({ length: totalDays }).map((_, idx) => {
      const day      = idx + 1;
      const date      = new Date(viewDate.getFullYear(), viewDate.getMonth(
      const key        = dateKey(date);
      const dayData    = schedule[key] || { main:[], light:[], checked:{} };
      const main        = dayData.main  || [];
      const light        = dayData.light || [];
      const newL         = main.filter(e => e.stage === 0);
      const reps         = main.filter(e => e.stage > 0);
      const isToday      = key === todayKey;
      const isSel        = key === selected;
      const isPast       = date < today;
      const hasLight      = light.length > 0;
      const allChecked    = hasLight && light.every(e => (dayData.checked ||

  return (
    <div key={key} onClick={() => setSelected(isSel ? null : key)}
      style={{
        background: isSel ? "#1a1208" : isToday ? "#fff3e0" : "#fff",
        border:`2px solid ${isSel ? "#c62828" : isToday ? "#ffa726" :
        borderRadius:8, padding:"7px 6px", cursor:"pointer",
        minHeight:82, opacity: isPast && !main.length && !light.lengt
        display:"flex", flexDirection:"column", gap:3,
        position:"relative",
      }}>

    {/* Light rep indicator dot */}
    {hasLight && (
      <div style={{
        position:"absolute", top:5, right:5,
        width:7, height:7, borderRadius:"50%",
        background: allChecked ? "#7c3aed" : "#ddd6fe",
        border:"1px solid #7c3aed",
      }} />

```

```

    })

    { /* Day number */
    <div style={{ fontSize:12, fontFamily:"'Courier New',monospace"
      color: isSel ? "#f5f0ea" : isToday ? "#e65100" : "#8c7f6e" }}

    { /* Main slot pips */
    <div style={{ display:"flex", gap:2 }}>
      {Array.from({length:6}).map((_,i) => {
        const e = main[i];
        const c = e ? mainColor(e.stage) : null;
        return <div key={i} style={{ flex:1, height:4, borderRadius
          }}}
      </div>

    { /* Summary */
    {(main.length > 0 || light.length > 0) && (
      <div style={{ display:"flex", flexDirection:"column", gap:2,
        {newL.length > 0 && (
          <div style={{ fontSize:10, color: isSel ? "#a5d6a7" : "#2
            ★ L{newL.map(e=>e.lessonId).join(",")}
          </div>
        )}
        {reps.length > 0 && (
          <div style={{ fontSize:10, color: isSel ? "#aaa" : "#8c7f
            ♡ {reps.length} rep{reps.length>1?"s":""}
          </div>
        )}
        {light.length > 0 && (
          <div style={{ fontSize:10, color: isSel ? "#c4b5fd" : "#7
            ◇ {light.length/2|0} light
          </div>
        )}
      </div>
    )}
    </div>
  );
  }}}
</div>

{ /* Legend */
<div style={{ display:"flex", gap:14, marginTop:14, flexWrap:"wrap" }}>
  {MAIN_STAGE_LABELS.map((lbl,i) => {
    const c = mainColor(i);
    return (
      <div key={lbl} style={{ display:"flex", alignItems:"center", gap:
        <div style={{ width:9, height:9, borderRadius:"50%", background

```

```

        <span style={{ fontSize:11, color:"#8c7f6e", fontFamily:"'Couri
    </div>
    );
  }}}
  {LIGHT_STAGE_LABELS.map(lbl => (
    <div key={lbl} style={{ display:"flex", alignItems:"center", gap:5
      <div style={{ width:9, height:9, borderRadius:"50%", background:"
        <span style={{ fontSize:11, color:"#7c3aed", fontFamily:"'Courier
      </div>
    )}}
  </div>
</div>

{/* — Side panel — */}
<div style={{ width:270, flexShrink:0, background:"#fff", borderLeft:"1px

  {!selected ? (
    <div style={{ color:"#8c7f6e", fontSize:13, marginTop:8 }}>← Click a
  ) : (
    <>
      {/* Date */}
      <div>
        <div style={{ fontSize:11, letterSpacing:3, color:"#8c7f6e", marg
        <div style={{ fontSize:16, color:"#1a1208" }}>
          {(() => { const d = new Date(selected); return `${d.getDate()}
        </div>
        <div style={{ fontSize:12, color:"#8c7f6e", marginTop:1 }}>{selMa
        <div style={{ display:"flex", gap:3, marginTop:7 }}>
          {Array.from({length:6}).map((_,i) => {
            const e = selMain[i]; const c = e ? mainColor(e.stage) : null
            return <div key={i} style={{ flex:1, height:6, borderRadius:3
              }}}
        </div>
      </div>
    </div>

    {/* Add button */}
    {selCanAdd ? (
      <button onClick={() => addLessons(selected)} style={{ background:
        + Add Lessons {nextLesson} & {nextLesson+1}
      </button>
    ) : selected && (
      <div style={{ background:"#fff3e0", border:"1px solid #ffcc02", b
        {(() => {
          const day = schedule[selected] || { main:[] };
          const newOnDay = day.main.filter(e=>e.stage===0).length;
          const total    = day.main.length;
          const tKey      = dateKey(addDays(new Date(selected),1));

```



```

        const tCount    = (schedule[tKey]?.main||[]).length;
        if (newOnDay >= MAX_NEW) return "Max 2 new lessons already se
        if (total > MAX_SLOTS-2) return "Day is full (6 slots)";
        return "Tomorrow too full for J1";
      })())
    </div>
  })

```

```

  { /* Main reps */
  { selMain.length > 0 && (
    <div>
      <div style={{ fontSize:11, letterSpacing:2, color:"#8c7f6e", ma
      <div style={{ display:"flex", flexDirection:"column", gap:5 }}>
        { selMain.map((e,i) => {
          const c = mainColor(e.stage);
          return (
            <div key={i} style={{ display:"flex", alignItems:"center"
              <span style={{ fontFamily:"'Courier New',monospace", fo
                {e.stage===0 ? `Lesson ${e.lessonId}` : `Rep L${e.les
              </span>
              <span style={{ fontSize:10, color:c.text, background:"r
                {MAIN_STAGE_LABELS[e.stage]}
              </span>
            </div>
          );
        })}
      </div>
    </div>
  })

```

```

  { /* Light reps - checkable, deduplicated by pair */
  { selLight.length > 0 && (
    <div>
      <div style={{ fontSize:11, letterSpacing:2, color:"#7c3aed", ma
      <div style={{ display:"flex", flexDirection:"column", gap:5 }}>
        { /* Group by lightStage, show one checkbox per pair */
        { [...new Set(selLight.map(e=>e.lightStage))].map(ls => {
          const pair    = selLight.filter(e=>e.lightStage===ls);
          const ids      = pair.map(e=>e.lessonId);
          const checkKey = `${ids[0]}-${ls}`;
          const isDone   = !!selChecked[checkKey];
          return (
            <div key={ls} onClick={() => toggleCheck(selected, ids[0]
              style={{ display:"flex", alignItems:"center", gap:10, b
                borderRadius:6, padding:"8px 10px", cursor:"pointer",
                border:`1px solid ${isDone ? "#7c3aed" : lightColor.b
              { /* Checkbox */

```

```

        <div style={{ width:18, height:18, borderRadius:4, border:1px solid #7c3aed, background: isDone ? "#7c3aed" : "transparent", flexS
            display:"flex", alignItems:"center", justifyContent:"center"
            {isDone && <span style={{ color:"#fff", fontSize:11,
        </div>
    <div style={{ flex:1 }}>
        <div style={{ fontSize:12, color:"#7c3aed", fontFamily:'monospace' }}>
            L{ids.join(" & ")}
        </div>
        <div style={{ fontSize:10, color:"#a78bfa", marginTop:5 }}>
            {LIGHT_STAGE_LABELS[ls]} review {isDone ? "✓ done" : ""}
        </div>
    </div>
</div>
);
}}}
</div>
</div>
))
</>
)}
</>
)}

<div style={{ flex:1 }} />

{/* Progress */}

<div>
    <div style={{ fontSize:11, letterSpacing:2, color:"#8c7f6e", marginBottom:10 }}>
        <div style={{ background:"#e0d9cf", borderRadius:999, height:7, overflow:hidden }}>
            <div style={{ height:"100%", borderRadius:999, background:"linear-gradient(to right, transparent 49%, #8c7f6e 49%, #8c7f6e 51%, transparent 51%)"}}>
            </div>
        <div style={{ display:"flex", justify-content:"space-between", margin-top:5 }}>
            <span style={{ fontSize:11, color:"#8c7f6e" }}>{totalLessons}</span> done</div>
            <span style={{ fontSize:11, color:"#8c7f6e" }}>{Math.max(0,250-totalLessons)}</span>
        </div>
    </div>

    </div>

{/* Light progress */}
{totalLight > 0 && (
    <div>
        <div style={{ fontSize:11, letterSpacing:2, color:"#7c3aed", margin-bottom:10 }}>
            <div style={{ background:"#ede9fe", borderRadius:999, height:7, overflow:hidden }}>
                <div style={{ height:"100%", borderRadius:999, background:"#7c3aed" }}>
                </div>
            <div style={{ display:"flex", justify-content:"space-between", margin-top:5 }}>
                <span style={{ fontSize:11, color:"#7c3aed" }}>{doneLight}</span> done</div>
                <span style={{ fontSize:11, color:"#7c3aed" }}>{totalLight-doneLight}</span>
            </div>
        </div>
    )}

```

```

    </div>
  )}

  { /* Undo */
  {nextLesson > 1 && (
    <button onClick={undoLast} style={{ background:"none", color:"#8c7f6e
      ↵ Undo L{nextLesson-2} & L{nextLesson-1}
    </button>
  )}

  { /* Reset */
  <button onClick={() => {
    if (confirm("Reset ALL data? Cannot be undone.")) {
      setSchedule({}); setNextLesson(1); setSelected(null);
      try { localStorage.removeItem(STORAGE_KEY); } catch(e) {}
    }
  }} style={{ background:"none", color:"#ccc", border:"1px solid #eee", b
    Reset all data
  </button>
  </div>
</div>

  <style>{`* { box-sizing:border-box; } button:hover { opacity:0.85; }`}</sty
</div>
);
}

const NAV = { background:"none", border:"1px solid #e0d9cf", borderRadius:6, padd

```